MOBILE APPLICATION DEVELOPMENT USING ANDROID

ASSIGNMENT – 5

Solved Assignment

Submitted By: Shreya Paliwal
Course: Bachelor of Computer Applications (BCA)
Subject: Mobile Application Development using Android

Department of Computer Science & Applications
COER University

Q1. Explain the concept of testing Android applications. Discuss the different types of testing (Unit Testing, Instrumentation Testing, UI Testing) and the tools used for each.

Android application testing is the process of checking an application to make sure that it works correctly, is reliable, performs well, and provides a good user experience. Testing helps identify errors before an app is released to users. It can be performed manually or automatically.

1. **Unit Testing:** Unit testing checks small, individual parts of an application, such as a method, class, or business-logic function. It is generally performed without launching the complete Android application. It helps developers find logic errors early. Common tools include **JUnit** for writing test cases and **Mockito** for creating mock objects and isolating the unit being tested.

2. **Instrumentation Testing:** Instrumentation tests run on an Android device or emulator and can interact with the Android framework. They are useful for testing components that need a real Android environment, such as activities, databases, and services. Common tools include **AndroidX Test** and **Espresso**; **UI Automator** can also test interactions across applications.

3. **UI Testing:** UI testing checks the screens and controls that users interact with. It verifies that buttons, text fields, menus, navigation, and displayed messages behave as expected. Espresso is commonly used for testing an app's own UI, while UI Automator is useful for system UI and cross-app interactions.

Conclusion: Unit tests check program logic, instrumentation tests check Android-dependent behaviour, and UI tests check the user interface. A combination of these tests improves application quality.

Q2. Explain the process of publishing an Android application. Discuss the key steps involved in preparing and releasing an app on the Google Play Store.

Publishing an Android application means preparing the final version of an app and making it available to users through the Google Play Store. The main steps are:

• **Complete development and testing:** Finish the app, fix known bugs, and test it on different screen sizes and Android versions.

• **Prepare the release build:** Remove debugging code, check app performance, configure the application ID, version code, and version name, and ensure that the release configuration is correct.

• **Create a signed release:** Generate a signed Android App Bundle (AAB), which is the preferred publishing format for Google Play. Protect the signing key and keep secure backups.

• **Prepare the Play Console listing:** Create or access a Google Play Console developer account, create the app entry, and add the app name, description, screenshots, app icon, category, and contact details.

• **Complete policy and content declarations:** Provide the required privacy policy, Data safety information, content rating, target audience details, and app access instructions where applicable. Request only necessary permissions.

• **Upload and review:** Upload the AAB, resolve any Play Console warnings, configure countries, pricing, and testing tracks, and submit the release for Google's review.

• **Release and monitor:** After approval, publish the app. Monitor crashes, Android vitals, user feedback, and reviews. Release updates when improvements or fixes are needed.

The exact Play Console requirements can change, so developers should follow the instructions shown in their Play Console account before submitting a release.

Q3. What are Android Preferences? Explain the use of SharedPreferences with a suitable example for storing and retrieving simple data.

Android Preferences are a way to store small amounts of application data as key-value pairs. They are useful for saving simple settings such as a user's name, theme choice, language, or login preference. **SharedPreferences** provides an API for storing and retrieving this data. The data remains available when the app is closed and opened again.

Example in Java: The following code saves a user's name and reads it later.

```
// Save a value
SharedPreferences preferences =
    getSharedPreferences("MyPrefs", MODE_PRIVATE);

SharedPreferences.Editor editor = preferences.edit();
editor.putString("user_name", "Shreya");
editor.apply();

// Retrieve the saved value
String name = preferences.getString("user_name", "User");

Toast.makeText(this, "Welcome " + name,
        Toast.LENGTH_LONG).show();
```

Explanation: The file named MyPrefs stores the preferences. The key user_name identifies the value. putString() stores a string, getString() retrieves it, and the second argument of getString() is the default value if no value has been saved. apply() saves changes asynchronously. SharedPreferences is intended for small, simple values, not large files or sensitive secrets.

Q4. Explain how Application Resources are managed in a hierarchy in Android Studio. Discuss the structure and purpose of the res/ folder.

Android separates application resources from Java or Kotlin source code. Resources are stored mainly in the **res/** directory inside the app module. Android Studio and the Android build system process these files and generate resource identifiers that the application can access through the R class.

Resources are organised by type in subfolders. For example, layouts are placed in layout/, images in drawable/ or mipmap/, and text values in values/. Android can also select alternative resources using qualified directory names, such as values-hi/ for Hindi text or layout-land/ for landscape layouts. The system chooses the best matching resource for the device configuration, such as language, screen size, or orientation.

Purpose of res/: It keeps reusable content separate from application logic, supports multiple languages and device configurations, and makes it easier to update the app's appearance without changing program code.

Q5. Discuss the different types of resources used in Android applications (strings, drawables, layouts, styles, dimensions), giving an example of each.

1. **Strings:** Text shown in the application is stored in res/values/strings.xml. This supports reuse and translation.

```
<resources>
    <string name="app_name">My Application</string>
    <string name="welcome">Welcome to the app</string>
</resources>
```

2. **Drawables:** Images and drawable XML files are stored in res/drawable/. They are used for icons, backgrounds, shapes, and other graphics. Example: res/drawable/logo.png or a shape drawable XML file.

3. **Layouts:** Layout XML files are stored in res/layout/. They define the structure of an activity or screen, including views such as TextView, Button, and EditText.

```
<!-- res/layout/activity_main.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
```

```
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/welcome" />
</LinearLayout>
```

4. **Styles:** Styles are defined in res/values/styles.xml (or themes.xml in newer project templates). They describe reusable appearance properties such as text size, colour, and padding.

```
<style name="HeadingText">
    <item name="android:textSize">22sp</item>
    <item name="android:textStyle">bold</item>
</style>
```

5. **Dimensions:** Dimension values are stored in res/values/dimens.xml. They help maintain consistent spacing and sizes. Use dp for layout dimensions and sp for text sizes.

```
<resources>
    <dimen name="screen_padding">16dp</dimen>
    <dimen name="title_text_size">20sp</dimen>
</resources>
```

These resources make Android applications easier to maintain, customise, and adapt to different devices.

Q6. Discuss common security threats and hacking techniques that target Android applications, with suitable examples.

Android applications may face different security threats when data, communication, or application code is not properly protected. Common threats include:

• **Insecure data storage:** Sensitive information may be saved in plain text in files, databases, logs, or preferences. For example, storing a password without protection can expose it if the device or app data is compromised.

• **Insecure network communication:** If an app sends data without properly protected HTTPS connections or fails to validate certificates, attackers on an unsafe network may intercept or modify information.

• **Reverse engineering:** Attackers may decompile an APK to inspect its code and discover hard-coded API keys, business logic, or other weaknesses.

• **Code tampering and repackaging:** An attacker may modify an application and distribute a repackaged version containing unwanted or harmful behaviour.

• **Phishing and fake applications:** A malicious app or website may imitate a trusted service to trick users into revealing login details or personal information.

• **Excessive permissions:** An application that requests unnecessary access to contacts, location, microphone, or files increases the impact of misuse.

• **Component exposure:** Exported activities, services, or broadcast receivers without suitable access controls may allow other apps to invoke functions unexpectedly.

For example, a poorly designed app that sends login details over an unprotected connection may expose user credentials. Developers should use secure storage, encrypted network communication, correct component permissions, and regular security testing.

Q7. Explain the best practices that should be followed to secure an Android application and protect user data.

• **Use HTTPS:** Send data through secure network connections and configure certificate validation correctly.

• **Protect sensitive data:** Avoid storing passwords, tokens, or personal information in plain text. Use Android Keystore-backed cryptographic keys and appropriate secure storage designs.

• **Follow least privilege:** Request only the permissions that are essential and explain why they are needed.

• **Validate input:** Check and validate user input to reduce risks such as injection, malformed data, and unexpected application behaviour.

• **Secure application components:** Set exported status carefully, protect sensitive components with permissions, and avoid exposing unnecessary interfaces.

• **Keep dependencies updated:** Use maintained Android libraries and SDK versions, and fix known security vulnerabilities.

• **Do not hard-code secrets:** Avoid placing private API keys, passwords, or encryption keys directly in source code.

• **Enable code shrinking and obfuscation:** Use R8 for release builds where appropriate. This makes reverse engineering harder, but it does not replace proper security controls.

• **Test security regularly:** Review code, check dependencies, test authentication and authorisation, and investigate crashes or suspicious behaviour.

• **Respect user privacy:** Collect only necessary data, explain data use, provide a privacy policy where required, and delete data when it is no longer needed.

Following these practices reduces security risks and helps maintain user trust.

Q8. What is Application Signing in Android? Explain the role of the keystore and APK/App Bundle signing in the publishing process.

Application signing is the process of digitally signing an Android application with a private key. Android uses the signature to verify the app's identity and to help ensure that updates come from the same authorised developer. A signed release is required for distribution.

Keystore: A keystore is a file or secure key store that contains the signing key and its certificate information. The developer must protect the keystore and private key because losing them or exposing them can create serious problems for app updates and security.

APK signing: An APK distributed directly to users must be signed. Android verifies the signature when the package is installed or updated.

Android App Bundle (AAB) signing: A developer uploads a signed AAB to Google Play. With Play App Signing enabled, the upload key authenticates the upload, while Google Play manages the app-signing key used to sign APKs delivered to users.

Importance: Signing establishes app identity, supports trusted updates, and helps protect the integrity of distributed application packages. Signing does not by itself encrypt all app data or guarantee that the app is free from vulnerabilities.

Q9. Explain the Android Permissions system. Discuss its role in maintaining application security and user privacy.

The Android Permissions system controls an application's access to restricted device features and user data, such as the camera, microphone, contacts, location, and storage. It gives users control over sensitive access and limits what an application can do.

Permissions are declared in the AndroidManifest.xml file. Some permissions are granted automatically because they present lower risk, while dangerous permissions require runtime approval from the user. The app should request runtime permissions when the feature is needed and explain the reason clearly. Users can review or revoke permissions in device settings.

Example manifest declaration:

```
<uses-permission
    android:name="android.permission.CAMERA" />
```

Before using a protected feature, the application should check whether the permission has been granted and request it if necessary. If the user denies permission, the app should handle that decision gracefully and avoid repeatedly pressuring the user.

Role in security and privacy: Permissions reduce unauthorised access to sensitive information, make app behaviour more transparent, and allow users to decide which capabilities an app may use. Developers should follow the principle of least privilege and request only permissions required for the app's core functionality.

Q10. Coding Question: Create a simple Android application that uses SharedPreferences to store a user's name the first time the app is launched, and displays a personalized greeting message on every subsequent launch. Also write the basic steps required to test and publish this application.

Objective: Save the user's name the first time the application runs. On later launches, retrieve the saved name and display a personalised greeting.

activity_main.xml (simple screen with a name field and button):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <EditText
        android:id="@+id/nameInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name" />

    <Button
        android:id="@+id/saveButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save Name" />

    <TextView
        android:id="@+id/greetingText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:paddingTop="16dp" />
</LinearLayout>
```

MainActivity.java:

```
package com.example.sharedprefapp;

import android.content.SharedPreferences;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private static final String PREF_NAME = "UserPreferences";
    private static final String KEY_NAME = "user_name";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        EditText nameInput = findViewById(R.id.nameInput);
        Button saveButton = findViewById(R.id.saveButton);
        TextView greetingText = findViewById(R.id.greetingText);

        SharedPreferences prefs =
                getSharedPreferences(PREF_NAME, MODE_PRIVATE);
```

```
        // Check whether a name was saved during an earlier launch.
        String savedName = prefs.getString(KEY_NAME, null);

        if (savedName != null) {
            greetingText.setText("Welcome back, " + savedName + "!");
            nameInput.setVisibility(android.view.View.GONE);
            saveButton.setVisibility(android.view.View.GONE);
        } else {
            greetingText.setText("Please enter your name.");

            saveButton.setOnClickListener(v -> {
                String name = nameInput.getText().toString().trim();

                if (!name.isEmpty()) {
                    prefs.edit().putString(KEY_NAME, name).apply();
                    greetingText.setText("Welcome, " + name + "!");
                    nameInput.setVisibility(android.view.View.GONE);
                    saveButton.setVisibility(android.view.View.GONE);
                } else {
                    nameInput.setError("Please enter your name");
                }
            });
        }
    }
}
```

Working: When the app starts, it checks SharedPreferences for the key user_name. If a name exists, the app displays “Welcome back” and hides the input controls. If no name exists, the user can enter a name and tap Save Name. The app stores the name using apply(). On the next launch, the saved name is retrieved and shown.

Basic testing steps:

• Run the application on an Android emulator or physical device.

• On the first launch, confirm that the app asks for a name and does not display a saved greeting.

• Enter a name and tap Save Name. Confirm that the personalised greeting appears.

• Close the app completely and launch it again. Confirm that the saved name is retrieved and the returning-user greeting appears.

• Test empty input, app restart, and different device or Android-version configurations. Check Logcat for errors.

Basic publishing steps:

• Set the final application ID, version code, and version name; test the release build.

• Generate a signed Android App Bundle (AAB) using a protected signing key.

• Create the app entry in Google Play Console and provide the store listing, screenshots, app icon, and required declarations.

• Complete the Data safety, content rating, privacy policy, and permission-related requirements that apply to the app.

• Upload the AAB to a testing track, test the release, and fix any reported problems.

• Submit the release for review and publish it after approval. Monitor crashes and user feedback after launch.

Result: The application stores the user's name on the first launch and displays a personalised greeting on every subsequent launch.

References

• Android Developers – App resources: https://developer.android.com/guide/topics/resources

• Android Developers – SharedPreferences: https://developer.android.com/reference/android/content/SharedPreferences

• Android Developers – Test apps: https://developer.android.com/training/testing

• Android Developers – App signing: https://developer.android.com/studio/publish/app-signing

• Google Play Console Help – Prepare and roll out a release: https://support.google.com/googleplay/android-developer/